

AutoSlim: Intelligent Automata Graph Optimization for Efficient Acceleration

Tiffany Yu and Rasha Karakchi (Advisor)

tyu@email.sc.edu, karakchi@cec.sc.edu

Department of Computer Science and Engineering,
University of South Carolina, Columbia, SC, USA

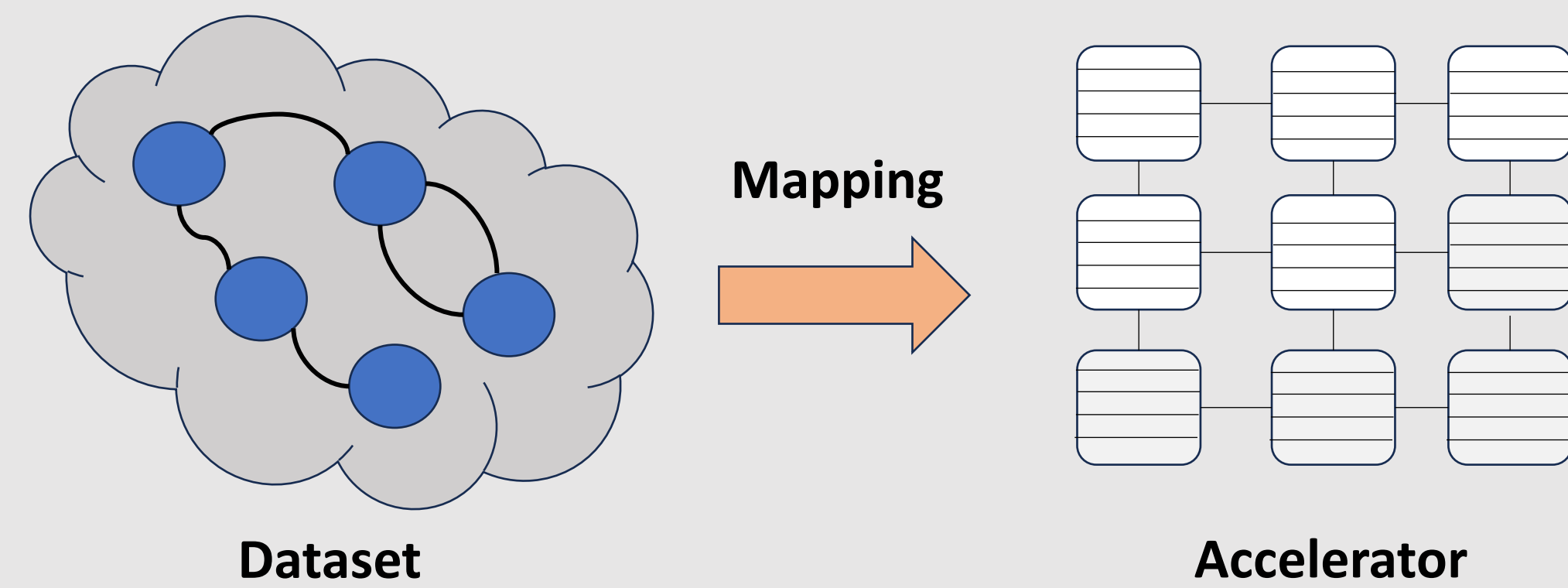


Symbolic Data

- **Symbolic data** (e.g., genomic sequences, logs) is widely used in bioinformatics, cybersecurity, and NLP.
- These workloads rely on **finite automata** for high-throughput pattern matching.

NAPOLY+

- A scalable FPGA overlay that supports **cost-aware NFA** execution using transition weights.
- It is two-dimensional array of processor elements where each processor element consist of RAM and logical gates.
- Datasets are mapping onto NAPOLY+ before starting the operation.
- Datasets represents labels/score attached to each edge.



Challenges in large datasets:

- Excessive memory from dense transitions
- Routing congestion
- Redundant/unreachable nodes

References

- Yu, Tiffany, et al. "ML-Based Automata Simplification for Symbolic Accelerators." arXiv preprint arXiv:2507.08751 (2025).
- Karbowniczak, Ryan, and Rasha Karakchi. "Optimizing Sequence Alignment with Scored NFAs." arXiv preprint arXiv:2501.02162 (2025). Run on existing tool to eliminate the redundancy

Acknowledgment

This work is supported by Magellan Scholar, Magellan Apprentice for Capstone Scholar and McNair Fellowship at USC. The authors thank **Rye Stahle-Smith** and **Darssan Eswaramoorthi** for their assistance with hardware testing and experimental setup. The authors also thank **AMD University Program** for providing devices for this project.

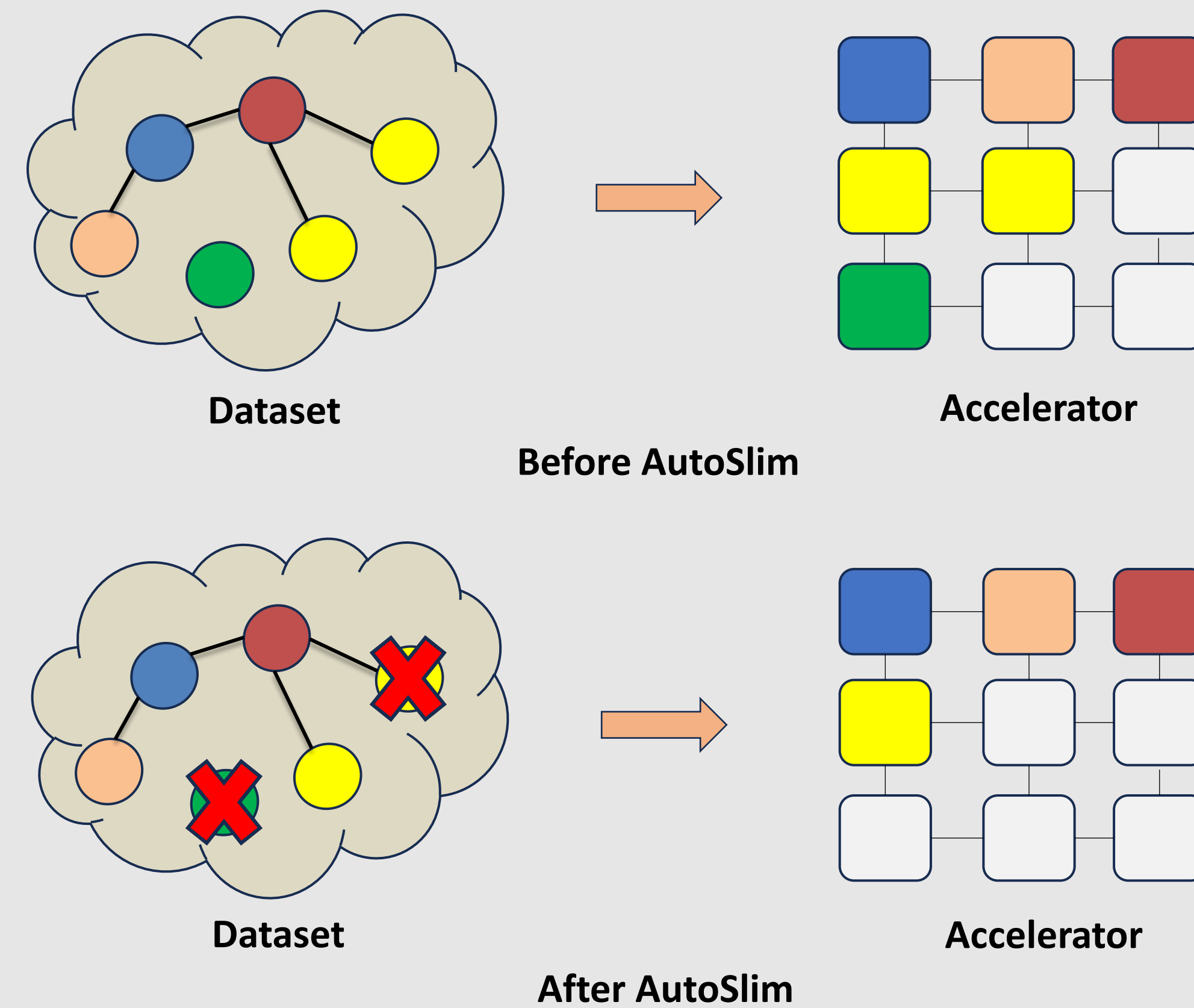
Scan to visit
the Github
Repository



Proposed Solution

AutoSlim's graph optimization.

- Top: Redundant nodes (yellow) and unreachable states (green) increase PE and routing cost.
- Bottom: Pruned graph after AutoSlim removes redundancies.



Algorithm

Algorithm 1 ML-based Transition Pruning Pipeline

```
1: Input: Folder of XML/CSV files with transitions, threshold  $\theta$ 
2: Output: Pruned transition files and evaluation reports
3: procedure PRUNETRANSITIONS(inputFolder, outputFolder,  $\theta$ )
4:   for all XML files in inputFolder do
5:     Parse each transition ( $from, to, score$ )
6:     Update node degree and total node score
7:     Export data to CSV format
8:   end for
9:   Create labeled dataset  $D = \{(x_i, y_i)\}$ , where  $y_i = 1 \text{ if } [x_i > \theta]$ 
10:  Train Random Forest classifier on D
11:  for all CSV files do
12:    Classify each transition ( $x_i$ ) using the trained model
13:    Keep transitions with  $\hat{y}_i = 1$ 
14:    Save pruned transitions to output CSV
15:    Generate report with state/transition count and accuracy
16:  end for
17:  return Pruned files and evaluation reports
18: end procedure
```

System Implementation and Workflow

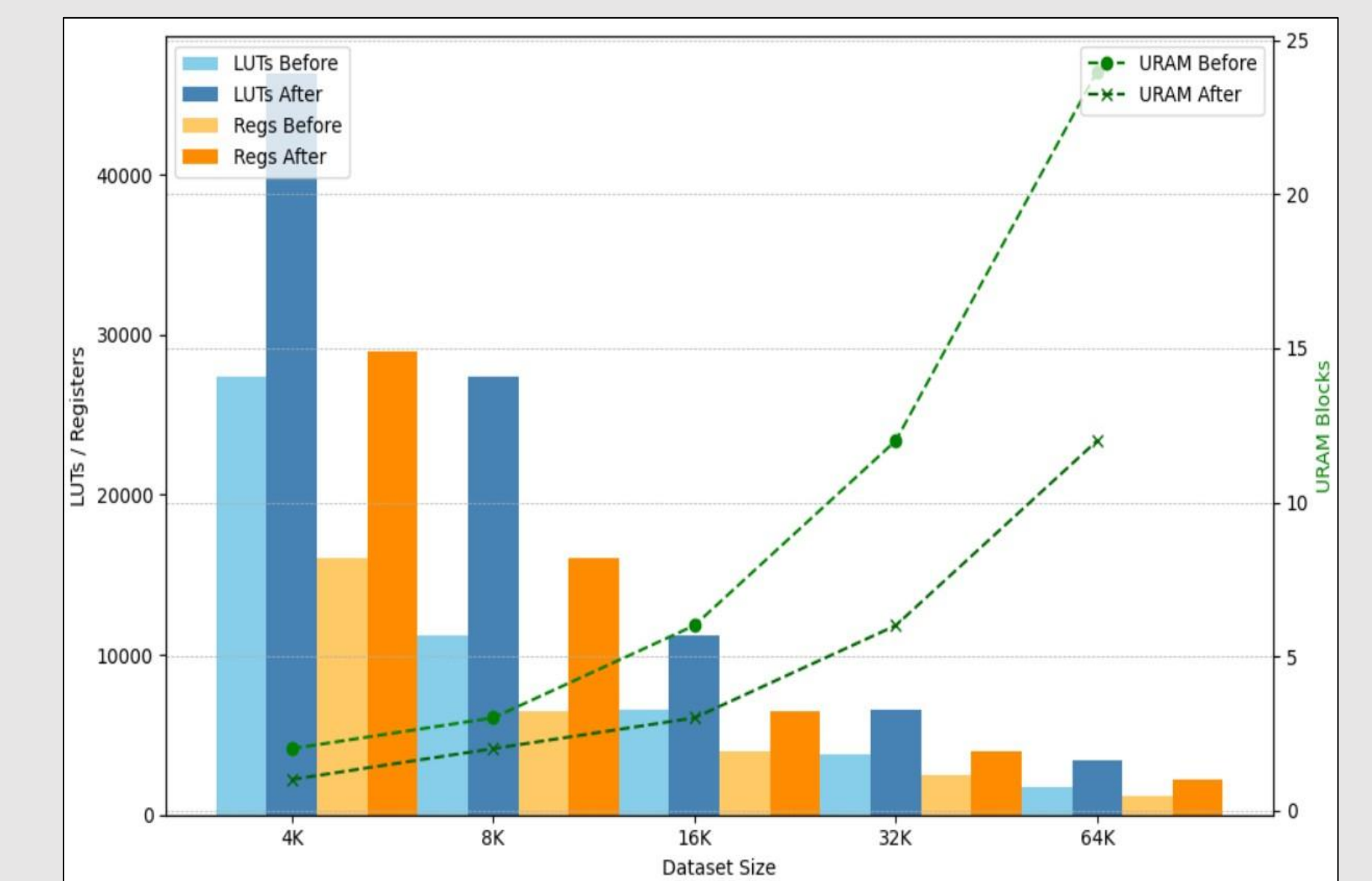
- **NAPOLY+ Re-implemented in Vivado HLS**
- Easy AutoSlim integration
- Latency, throughput, and resource analysis
- **Enhancements:**
- Supports various graph sizes & pruning levels
- Enables cycle-accurate simulation
- Pruned graphs → Config vectors → HLS-based NAPOLY+
- **AutoSlim Workflow:**
- End-to-End Flow: Symbolic input → AutoSlim → FPGA deployment → Performance analysis
- **Benefits:**
- Fewer states/transitions
- Lower hardware overhead (LUTs, BRAMs, PEs)
- Faster execution across datasets

Evaluation and Conclusion

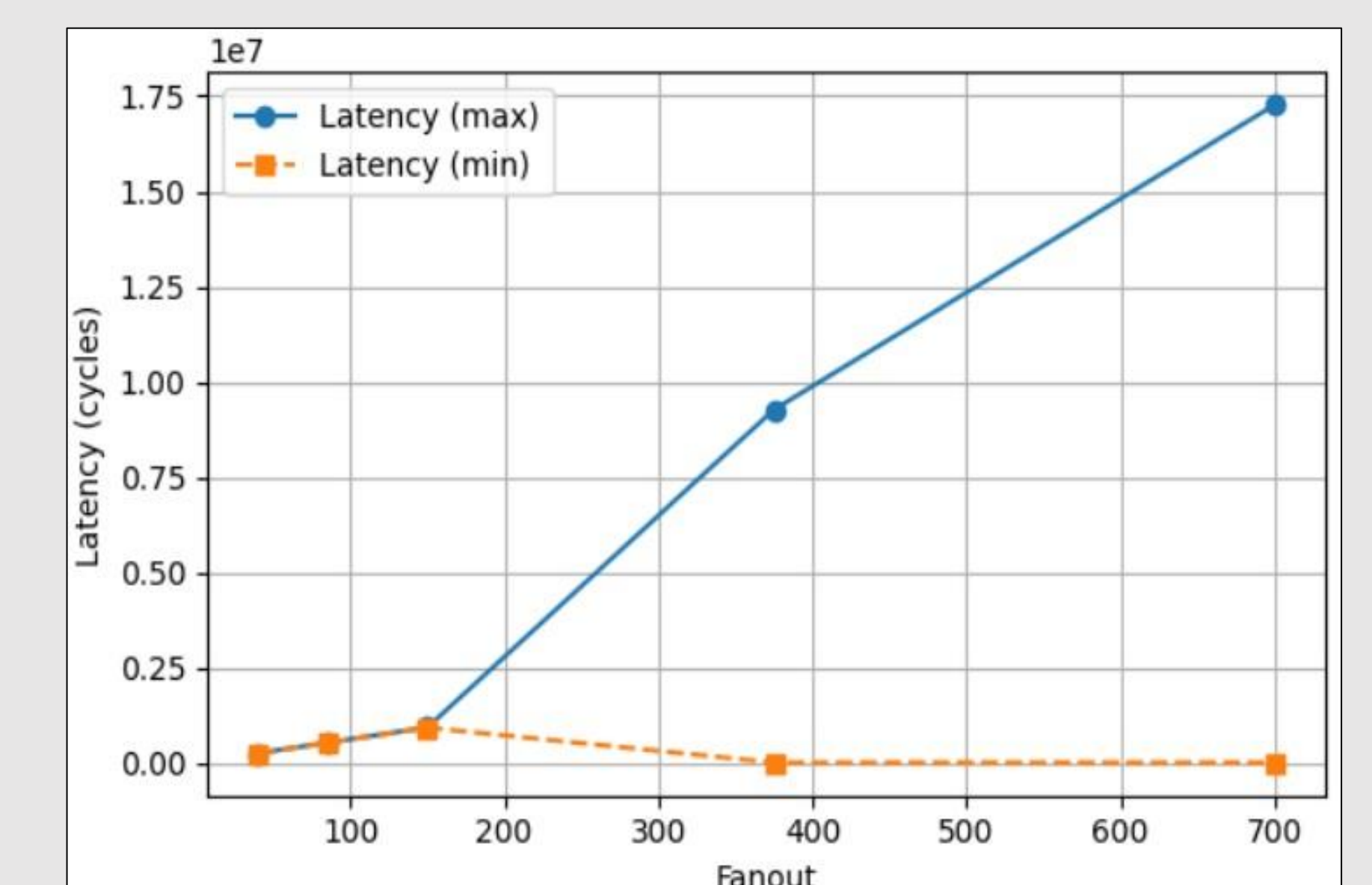
- **Execution Time:**
- Decreases with graph size due to increased pruning
- Larger graphs (16K–64K) show more sparsity and faster execution
- **Transition Reduction:**
- Over 30–40% total transition reduction
- Post-pruning transitions remain nearly flat across scales
- **Graph Sparsity:**
- Avg. transitions per node drop from ~180 to <5 (1K → 64K)
- Graphs become significantly simpler and more hardware-efficient
- **Hardware Deployment (ZCU104):**
- LUTs drop from 47K to 7K
- Registers drop from 30K to 2K
- URAM usage cut by 50%
- **Fanout Analysis:**
- High fanout increases LUTs, routing congestion, and max latency
- AutoSlim reduces fanout while preserving critical scoring paths

Results

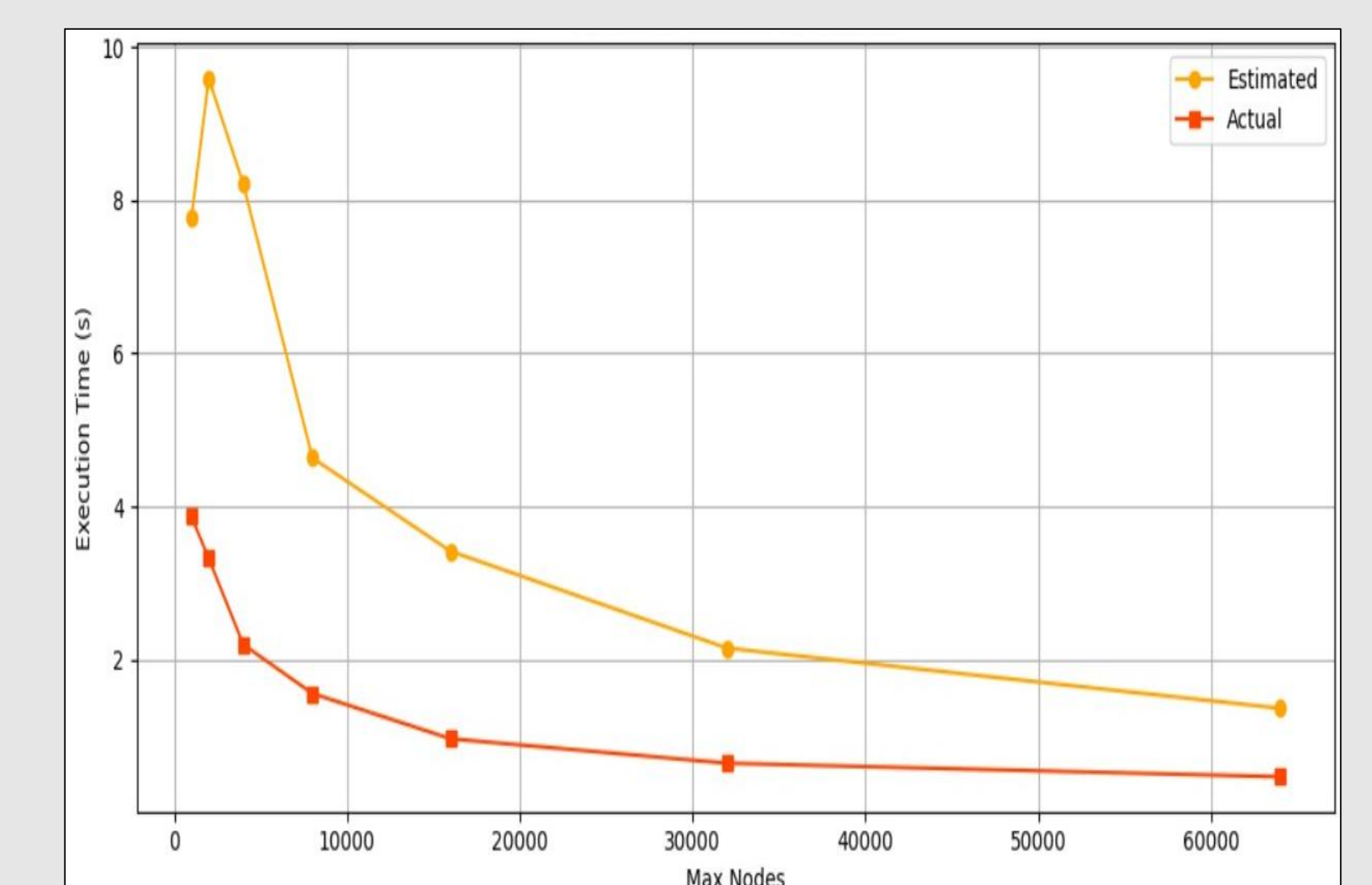
Number of LUTs, Registers, URAM in different size of datasets.



Max and Min number of Latency cycles according to number of Fanout.



Execution Time in secs VS the number of max nodes.



Execution Time in secs VS the different sizes of datasets.



UNIVERSITY OF
South Carolina